

基于 K-means 聚类算法的手写数字识别实验

刘厚毅 42250040

一、摘要：

本实验基于无监督学习中的 K-means 聚类算法，探索其在 MNIST 手写数字识别任务中的有效性。通过对图像数据进行标准化和 PCA 降维，减少维度对聚类性能的影响。实验对比了随机初始化、K-means++ 和密度初始化三种方式，使用准确率、轮廓系数、兰德指数和训练时间等指标进行评估。结果表明，密度初始化在聚类质量上表现最佳，但计算开销较大；随机初始化在充分迭代后获得略高的准确率。实验还对比了 K-medoids 算法在小样本集下的表现，并通过可视化手段辅助分析聚类效果，为无标签场景下的数字识别提供参考。

二、引言

手写数字识别作为计算机视觉的基础任务，在邮政编码识别、表单自动化处理等领域具有重要应用价值。MNIST 数据集作为包含 70,000 张 0-9 手写数字图像的基准数据集，常被用于验证机器学习算法的有效性。传统监督学习方法（如 SVM、CNN）虽能实现高精度分类，但其依赖大量标注数据的特性限制了在无标签场景下的应用。

本研究聚焦无监督学习范式，探索经典 K-means 聚类算法在 MNIST 数据上的数字识别能力。K-means 通过迭代优化簇中心实现数据分组，其无需标签的特性可降低数据标注成本，但面对图像数据的高维度特性及数字书写风格差异，算法在特征表达和簇间区分度方面面临挑战。本实验旨在通过特征降维（如 PCA）、聚类评估指标（轮廓系数、兰德指数）和可视化分析，系统验证 K-means 在无监督数字识别中的有效性，同时揭示其与监督方法的性能差异，为轻量化识别场景提供技术参考。

三、实验思路与理论支持

K-Means 算法是一种无监督学习，同时也是基于划分的聚类算法，一般用欧式距离作为衡量数据对象间相似度的指标，相似度与数据对象间的距离成反比，相似度越大，距离越小。算法需要预先指定初始聚类数目 k 以及 k 个初始聚类中心，根据数据对象与聚类中心之间的相似度，不断更新聚类中心的位置，不断降低类簇的误差平方和（Sum of Squared Error, SSE），当 SSE 不再变化或目标函数收敛时，聚类结束，得到最终结果。

[1] 如何确定聚类中心数 K ？

因为实验为数字识别，数字从 0 到 9 一共有 10 个，因此选择 $K=10$ 。

[2] 如何初始化聚类中心？

• 随机选择法：

最简单直接的方式，从数据集中随机挑选 K 个数据点作为初始聚类中心点。这种方法易于实现，但可能因随机性导致聚类结果不稳定，若初始点选得不好，可能使算法收敛到较差的局部最优解。

• K-means++初始化：

先初始化一个中心点 c_1 ；计算每个样本到最近中心点的距离，

$$d(x_i) = \operatorname{argmin} \operatorname{distance}(x_i, c_j)$$

选择下一个中心点时，基于 $d(x_i)$ 进行概率采样：

$$P(x_i) = \frac{d(x_i)}{\sum_{x \in X} d(x_i)}$$

即：距离越远的样本被选中的概率越高。

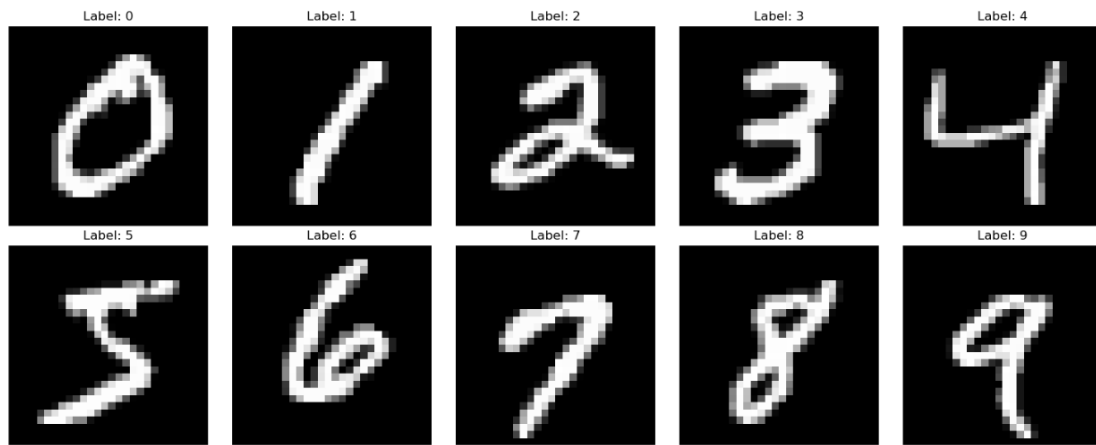
- **基于密度的初始化**

计算所有数据对象的密度，求出数据集的密度平均值，将大于密度平均值的数据对象作为高密度点集，从密度点集中选取密度值最大的数据对象作为第一个初始聚类中心，剩余聚类中心选取依据与前面选定的聚类中心的距离最大的原则进行。

理论上讲，后两种初始化方法的聚类效果会比随机中心点的好，我们也可以在实验中验证它们在此数据集上的表现如何，我们可以通过计算**轮廓系数（Silhouette Score）**和**调整兰德指数（Adjusted Rand Index）**来评估聚类质量，通过运行时间来比较收敛速度。

[3] 如何确定聚类中心和样本点的距离？（如何用特征向量表示图片？）

对于 MNIST 数据集的手写图像，每张图片都由 $28 \times 28 = 784$ 个像素点组成，每个像素的值范围是 0 到 255，表示灰度值（0 表示黑色，255 表示白色）。



我们可以把每张图像表示为一个特征向量，每个像素的灰度值成为向量的一个维度。因此，每张图像可以表示为一个 784 维的向量（ 28×28 像素展平后的结果）。这样我们就可以用这个 784 维的向量计算任意两个点之间的欧式距离

$$d(x_i, c_j) = \sqrt{\sum_{k=1}^{784} (x_{i,k} - c_{j,k})^2}$$

x_i 为样本向量

c_j 表示中心点向量

$x_{i,k}$, $c_{j,k}$ 分别是样本和中心点的第 k 个像素点的灰度值。

计算距离时，计算样本每一个像素点的灰度值与中心点相应像素点灰度值之间的差值，平方加总后开方得到两个点间的欧式距离。

[4] 如何设置终止条件

- **最大迭代次数**

设置一个最大迭代次数，当达到这个次数时，算法停止。

- **中心点变化阈值**

设置一个中心点变化的阈值，当中心点的变化小于这个阈值时，算法停止。

在具体实验中，我会使用 for, range 函数来改变迭代次数，观察结果准确率的变化情况，我们默认中心点变化阈值为 $1e-4$ (0.0001)，调整 tol 参数为 $1e-5$ 或 $1e-6$ ，观察准确率变化情况。

四、实验过程与结果（以 k-means++ 为例）

[1] 数据预处理：

首先将 DataFrame 转换为 NumPy 数组，再进行 Min-Max 归一化，最后划分测试集与训练集分别为 60000 个图像。

```
# 将 DataFrame 转换为 NumPy 数组
X = X.values

# Min-Max 归一化
X_normalized = X / 255.0

# 划分训练集和测试集
X_train, X_test = X_normalized[:60000], X_normalized[60000:]
y_train, y_test = y[:60000], y[60000:]
```

[2] 算法主程序：

使用 sklearn 库中的 KMeans 函数，

```
# 初始化 K-means
start_time = time.time()
kmeans = KMeans(n_clusters=10, init='k-means++', max_iter=300, tol=1e-4, random_state=42)
kmeans.fit(X_train)
end_time = time.time()
train_time = end_time - start_time
```

其中，n_clusters 表示聚类数量 k，我们预设为 10；

init 为指定初始化方法，可以设定为 random（随机初始中心）或 centroids 自定义；

max_iter 为最大迭代次数；

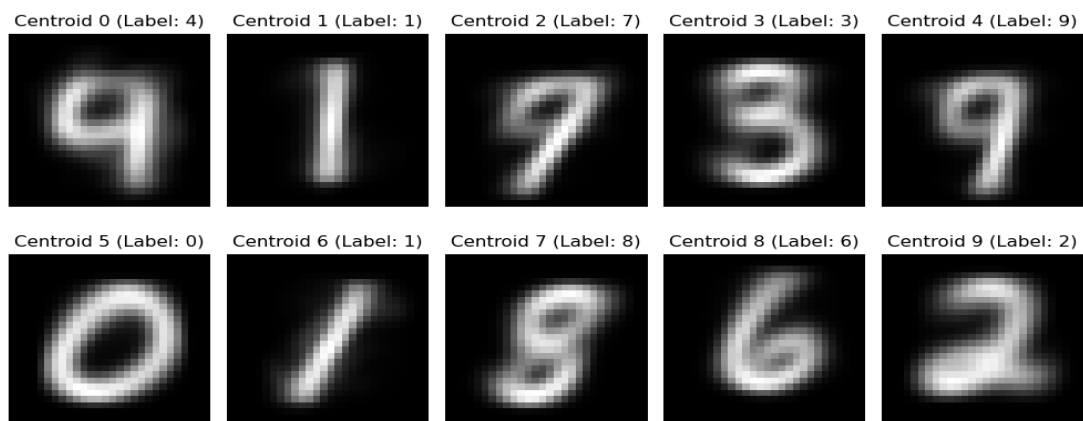
tol 为中心点变化阈值；

random_state 为随机种子，确保结果可以复现。

最后我还使用 time 函数记录程序运行时间，用于模型评估。

[3] 运行结果可视化与评估可视化

使用标签映射将聚类结果和真实结果一并表现出来，可以看出识别效果不是很好，0、4、9 混为一谈。

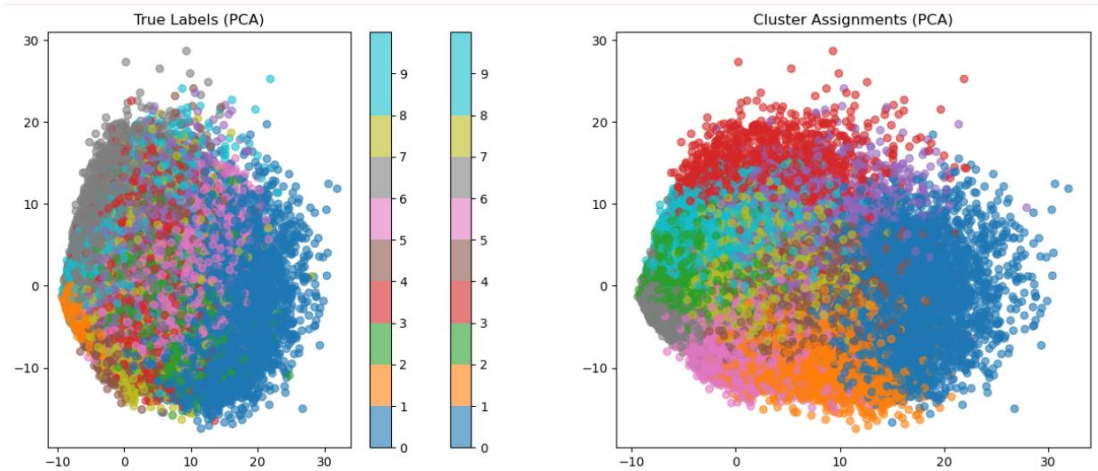


- 模型评估：**
 使用准确率、计算时间和轮廓系数对程序进行评估，得到以下结果：

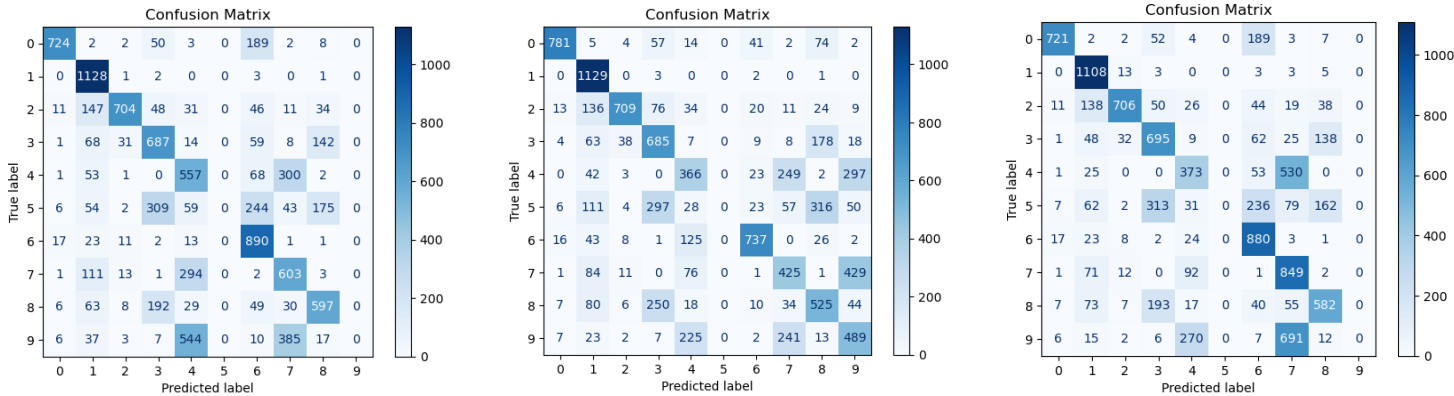
	Classification Accuracy	Training Time	Silhouette Score	Adjusted Rand Index
Random	0.59	24.61s	0.058	0.406
K-means++	0.58	13.96s	0.056	0.395
Density	0.59	54.68s	0.071	0.412

（兰德系数和轮廓系数范围[-1, 1]，越高越好）

- 聚类可视化：**
 使用 PAC 降维到 2D.



混淆矩阵：



（从左至右分别为 Random、K-means++、Density）

五、实验分析

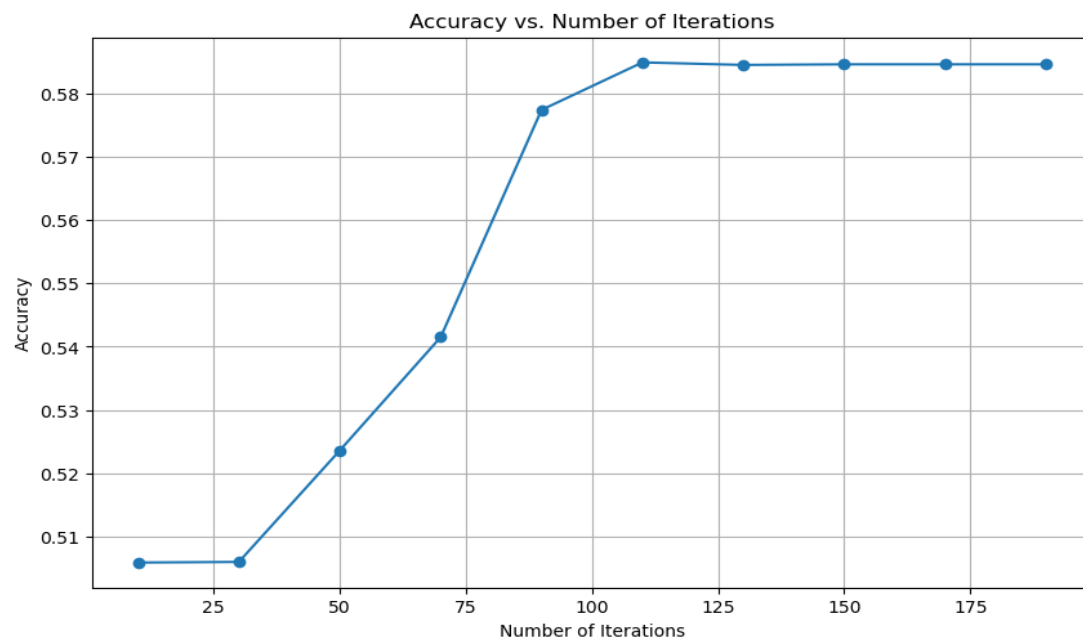
- 初始化聚类中心对聚类效果的影响：**
 理论上 k-means++的聚类效果会比随机初始化优秀，但是在本次实验中，充分迭代（max_iter=600）后的结果显示随机初始化的聚类效果更好，准确率为 0.59，略高于 k-means++的 0.58。初步推测可能是样本结构带来的影响——样本存在离群点距离过远，导致离群点的被选中的概率更大，降低了准确度。

在三种初始化方法中，基于密度的聚类效果最好，计算成本也最高。也侧面说明了初始化越复杂，聚类效果越好。

总而言之，初始化方法对聚类效果的影响巨大。

- **迭代次数对聚类效果的影响：**

选择 k-means++ 作为初始化中心点方法，通过调整最大迭代数，我得到了准确率随迭代次数增长而增长的趋势图：

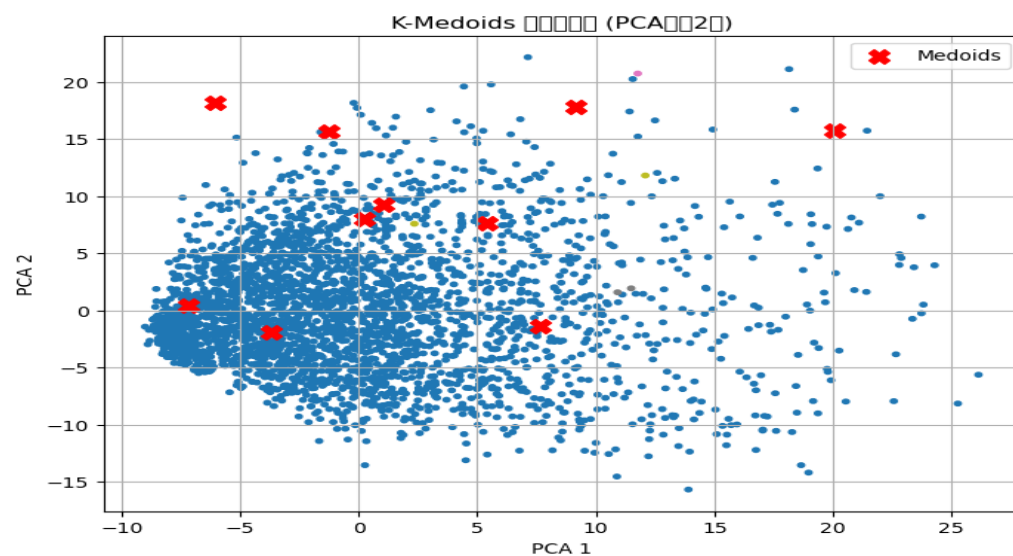


可以看出，准确率在迭代数 110 以上时变化不大，可能是算法已经收敛，K-means 算法的目标是找到使得簇内误差平方和（惯性）最小的聚类结果。当算法收敛时，中心点的位置和样本的分配不再变化，因此继续增加迭代次数不会改变聚类结果。

- **其他聚类方法的使用**

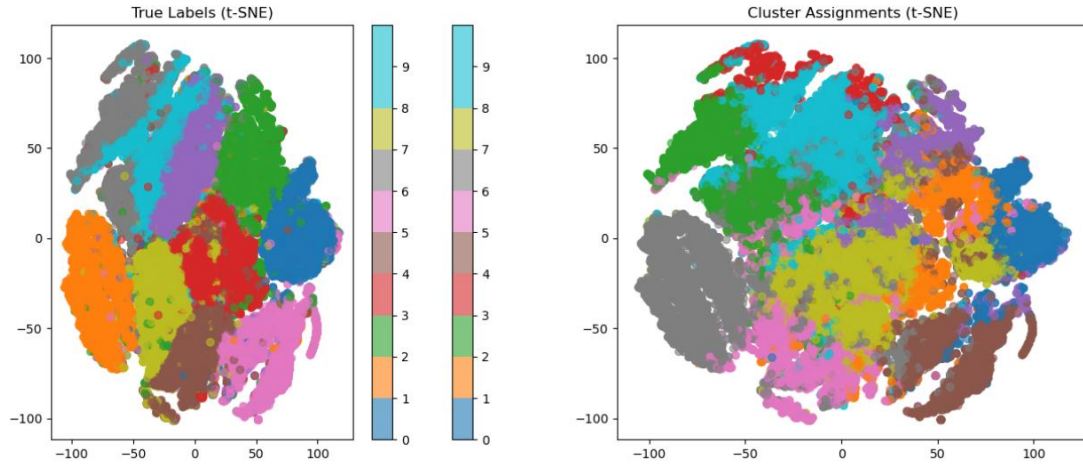
使用 k-medoids 聚类方法也可以实现手写数字识别，使用曼哈顿距离。

最后 K-medoids 算法得到的评估结果为：轮廓系数：0.6132276514236895，调整兰德指数： $-5.008451042030289 \times 10^{-5}$ ，准确率为 0.1173，因为此算法需要大量内存和算力，本实验只使用了 3000 个样本，迭代最多 15 此，因此聚类效果和识别效果都很差。



- 优化可视化: T-SNE 降维可视化

t-SNE (t-Distributed Stochastic Neighbor Embedding) 是一种用于降维和可视化的技术, 使用高斯分布来计算每个数据点与其邻居点的条件概率, 以衡量它们的相似性, 特别适合于高维数据的可视化。



附录

k-means++主程序代码

```
# 初始化 K-means
start_time = time.time()
kmeans = KMeans(n_clusters=10, init='k-means++', max_iter=300, tol=1e-4, random_state=42)
kmeans.fit(X_train)
end_time = time.time()
train_time = end_time - start_time
```

获取聚类结果和中心点

```
# 获取聚类结果
train_labels = kmeans.labels_
centroids = kmeans.cluster_centers_
```

```
# 标签映射
label_map = np.zeros(10, dtype=np.int8)
for i in range(10):
    mask = (train_labels == i)
    label_map[i] = mode(y_train[mask])[0]
```

附录 2

聚类可视化主代码

```
# 使用 PCA 降维到 2D
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)

# 可视化聚类效果
plt.figure(figsize=(10, 6))
plt.scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=train_labels, cmap='viridis', marker='.')
plt.title('Clustering Result in 2D')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.colorbar()
plt.show()
fig = plt.gcf()
```